

Laboratorio Sesión 08: Sistemas de trabajo colaborativo

En esta sesión vamos a ver algo sobre los sistemas de control de versiones, muy usados en el desarrollo de software libre... y de no libre (y de hecho también para la elaboración de documentos en L^AT_EX :-).

Un sistema de versiones, es, básicamente, un programa que a partir de una copia principal de un programa permite descargar copias locales y actualizarlas en el servidor central. Guarda además, un registro de cambios (de forma que estos pueden deshacerse), y detecta cuando dos o más personas han modificado las mismas líneas de un código de forma que se puede deshacer el error (aunque esto último debe hacerse a mano). En el fondo es algo muy simple pero ayuda mucho a organizar un trabajo entre varias personas (Wikipedia, en realidad, es la misma idea pero aplicada a texto).

8.1. Git

Existen muchos sistemas de control de versiones. Actualmente, uno de los mejores (a mi entender) y más usados es Git. Git fue creado por Linus Torvalds que ha dicho que el nombre (en inglés viene a significar estúpido) lo puso porque: *"I'm an egotistical bastard, and I name all my projects after myself. First 'Linux', now 'git'"*. La página de man describe git como *"the stupid content tracker"*.

En cualquier caso, git es un ejemplo de como surge y mejora el software libre. En sus inicios Linus usaba BitKeeper para desarrollar Linux. Sin embargo los propietarios decidieron retirar la licencia de libre uso debido a que, al parecer, algún ingeniero había desencryptado sus protocolos. Las reflexiones de Linus para desarrollar Git fueron las siguientes (transcritas de un correo suyo):

However, the SCMs I've looked at make this hard. One of the things (the main thing, in fact) I've been working at is to make that process really efficient. If it takes half a minute to apply a patch and remember the changeset boundary etc. (and quite frankly, that's fast for most SCMs around for a project the size of Linux), then a series of 250 emails (which is not unheard of at all when I sync with Andrew, for example) takes two hours. If one of the patches in the middle doesn't apply, things are bad bad bad.

Now, BK wasn't a speed demon either (actually, compared to everything else, BK is a speed demon, often by one or two orders of magnitude), and took about 10-15 seconds per email when I merged with Andrew. HOWEVER, with BK that wasn't as big of an issue, since the BK-BK merges were so easy, so I never had the slow email merges with any of the other main developers. So a patch-application-based SCM "merger" actually would need to be faster than BK is. Which is really really really hard.

So I'm writing some scripts to try to track things a whole lot faster. Initial indications are that I should be able to do it almost as quickly as I can just apply the patch, but quite frankly, I'm at most half done, and if I hit a snag maybe that's not true at all. Anyway, the reason I can do it quickly is that my scripts will not be an SCM, they'll be a very specific "log Linus" state" kind of thing. That will make the linear patch merge a lot more time-efficient, and thus possible.

(If a patch apply takes three seconds, even a big series of patches is not a problem: if I get notified within a minute or two that it failed half-way, that's fine, I can then just fix it up manually. That's why latency is critical-if I'd have to do things effectively "offline", I'd by definition not be able to fix it up when problems happen).

Con estos prerequisites, Linux llegó a las siguientes reglas de diseño:

1. Tomar CVS como un ejemplo de lo que no había que hacer: en caso de duda hacer exactamente lo contrario. Citando de nuevo a Linus:

For the first 10 years of kernel maintenance, we literally used tarballs and patches, which is a much superior source control management system than CVS is, but I did end up using CVS for 7 years at a commercial company (Transmeta) and I hate it with a passion. When I say I hate CVS with a passion, I have to also say that if there are any SVN (Subversion) users in the audience, you might want to leave. Because my

hatred of CVS has meant that I see Subversion as being the most pointless project ever started. The slogan of Subversion for a while was “CVS done right”, or something like that, and if you start with that kind of slogan, there’s nowhere you can go. There is no way to do CVS right.

Como podéis ver, Linus no es un gran fan de CVS y todos los sistemas que de él dependen.

2. Soportar el trabajo distribuido tal y como hacía BitKeeper (de nuevo cita de Linus):

BitKeeper was not only the first source control system that I ever felt was worth using at all, it was also the source control system that taught me why there’s a point to them, and how you actually can do things. So Git in many ways, even though from a technical angle it is very very different from BitKeeper (which was another design goal, because I wanted to make it clear that it wasn’t a BitKeeper clone), a lot of the flows we use with Git come directly from the flows we learned from BitKeeper.

3. El sistema debía tener un fuerte sistema de seguridad contra la corrupción de los datos, accidental o intencionada.
4. Debía dar un gran rendimiento.

El problema era que los primeros tres criterios eliminaban todos los sistemas de control de versiones ya existentes excepto “Monotone” y el cuarto los excluía todos. Así que Linus decidió escribir el suyo propio (que fue solo aplicable a su proyecto y poco usable con respecto a otros sistemas de control de versiones igual que Linux iba a ser un pequeño experimento que solo iba a correr en su ordenador).

Bueno, toda esta disertación sobre Git en realidad tiene un objetivo y es mostrar los beneficios de la filosofía de trabajo **KISS** que muchas veces vale la pena tener en cuenta.

Pregunta 8.1: ¿Qué significa KISS? ¿Qué filosofía de diseño plantea?

La principal característica diferencial de Git respecto a los sistemas anteriores es que no mantiene las diferencias de unas versiones respecto a otras sino que copia todos los ficheros del repositorio (con todas las versiones) cada vez que se produce una actualización. Esto hace que sea muy fácil (y rápido) trabajar en local, e incluso, desconectado. Además, todas las copias del repositorio son en realidad el repositorio completo así que es muy fácil restaurar y/o recuperar información.

Bueno, visto todo esto vamos a ver como funciona Git. Primero instalaros el paquete. A continuación, deberéis crear un repositorio. Para ello cread un directorio vacío y entrad en él. A continuación escribid:

```
git init
```

Si todo ha ido bien esto os creará un directorio llamado `.git` que contendrá la información sobre el repositorio. A continuación podéis mirar en que estado se encuentra el repositorio (`git status`) y crear un nuevo fichero (de la forma usual).

Pregunta 8.2: ¿Qué mensaje muestra `git status` referente al nuevo fichero?

El mensaje de `git status` nos dice que tenemos un nuevo fichero pero que no está en el repositorio. En git hay tres estados: cambios que “no existen” para git (untracked), cambios que están pendientes (staged) y cambios que están en el repositorio (committed). Si queremos añadir el nuevo fichero a los cambios pendientes tenemos que incluirlo con la orden:

```
git add <nombrefichero>
```

Fijaros que ahora el fichero ha pasado a estar pendiente de “envío” hacia el repositorio. Bien, si ya estamos seguros podemos incorporar el fichero al repositorio (en realidad la orden actualiza todos los cambios pendientes) con `git commit`. El problema de esta orden es

que nos pide un comentario con un editor de texto más o menos engorroso. Abortad (`^X`) y ejecutad el `commit` con la opción `-m` y un comentario entrecomillado.

Pregunta 8.3: *¿Para qué sirve el parámetro `-m`? ¿Qué información da `git` referente al nombre y correo del “committer”? ¿Cómo se soluciona?*

Podéis comprobar el nuevo estatus del repositorio. Probad a añadir algunos ficheros más al repositorio (individualmente o en grupo) y a continuación consultad la historia del repositorio mediante el comando `git log`. Fijaros que si modificáis algún fichero que ya habéis añadido (probad a hacerlo) `git` no guarda las modificaciones inmediatamente sino que nos avisa de que las ha habido y de que hay que hacer un nuevo `git add` para añadirlas a los cambios que serán enviados.

Pregunta 8.4: *¿Qué información muestra `git log`? ¿Y `git status`? ¿Cómo dice `git status` que se ha de volver a la versión sin cambios de un fichero si queremos descartar dichos cambios?*

También es muy útil la orden `git diff` que nos informa de los cambios que todavía no hemos “marcado” para incorporar al envío. Probad a modificar un fichero y ejecutad `git diff`.

Pregunta 8.5: *¿Cómo marca los cambios `git`?*

Si a continuación hacemos `add` del fichero con los cambios, estos desaparecen del `git diff` y aparecen entre la versión “staged” y la final y se pueden ver con `git diff --staged`. Otra opción útil es borrar ficheros de `git`, tanto del repositorio (`git rm fichero`) como del “stage” (`git reset fichero`) si decidimos que queremos olvidarnos de él. Probad a borrar alguno.

Pregunta 8.6: *¿`Git log` informa de los ficheros que se borran del “stage”? ¿Y de los del repositorio? ¿Y `git status`?*

Bueno, ahora que ya sabemos operar en local en `Git`, nos queda poder operar con un repositorio remoto. En realidad es muy fácil. Para hacerlo vais a trabajar de nuevo en parejas de parejas. Una pareja tendrá el repositorio original (servidor) y la otra se conectará como cliente. En realidad `git` permite muy fácilmente cambiar los roles ya que todas las copias del repositorio son en realidad el propio repositorio.

Lo primero que debéis hacer es crear un repositorio para compartir (antes habíais creado un repositorio de trabajo) en un nuevo directorio vacío:

```
git init --bare
```

La diferencia básica con la vez anterior (que indica la opción `--bare`) es que ahora este directorio no está pensado para trabajar directamente en él como antes sino para ser compartido por varios usuarios (esto imita el funcionamiento original de `CVS`). Si queréis trabajar con este repositorio desde el mismo ordenador, en vez de hacerlo directamente, iros a otro directorio nuevo y clonad el directorio `bare`:

```
git clone /camino/al/repositorio/git/bare
```

En el directorio nuevo, que será el de trabajo, ya podéis añadir ficheros con contenido, hacer el `commit` y enviarlos al repositorio que teníais vacío con `git push`. Aunque parece más difícil al principio ya que hemos tenido que crear dos directorios (el del repositorio y el de trabajo) ahora el repositorio se puede usar entre varios ordenadores. Para hacer esto deberéis aseguraros de tener activo el servidor `ssh` en la máquina servidor. A continuación, el sistema cliente ejecutará:

```
git clone usuario@pcservidor:/camino/al/repositorio/git/bare
```

Y ya está, el cliente tendrá una copia del repositorio en local con la que podrá trabajar. Es decir, para crear un servidor `git` basta con tener acceso `ssh` a una máquina, así de simple. En

realidad lo normal en proyectos de software libre es crearse una cuenta en servidores públicos tipo “GitHub” o “SourceForge”, pero también podéis gestionar muy fácilmente vuestro propio servidor (acabáis de configurar uno).

Bien, ahora que sabemos trabajar con repositorios remotos, lo normal es que a veces queramos incorporar cambios sin bajar todo el repositorio que es lo que hace `git clone`. Fijaros que como todos los repositorios son iguales lo que marca la diferencia de sentido (bajar ficheros del repositorio remoto al local o subir nuestras modificaciones) es sólo el ordenador en el que trabajamos, en realidad no hay repositorio central.

Empecemos por actualizar la información del repositorio local con lo que hay en el remoto: esto puede hacerse de dos formas: `git fetch origin` o `git pull origin`. La diferencia es que `fetch` nos bajará toda la información y a continuación nos dejará mezclar los cambios a nosotros (luego veremos algo de las ramas), mientras que `pull` mezcla directamente la rama principal remota con nuestra rama principal así que si sois novatos utilizad mejor `git pull`. `origin` es el nombre que tiene por defecto (lo pone el `clone`) el repositorio remoto a no ser que hayamos creado otro explícitamente con la orden `git remote add <nombre> <url>` (de hecho podemos tener varios repositorios remotos y trabajar con todos a la vez). La orden para subir nuestros cambios es la contraria: `git push origin master`.

Pregunta 8.7: *A continuación bajaros el repositorio principal (`git clone`) en dos directorios diferentes (uno la pareja “servidora” y otro la “cliente”), y modificad ambos el mismo fichero. ¿Os deja enviar (con la orden `git push origin master`) la modificación a ambos?*

Pregunta 8.8: *¿Cómo se puede corregir el error para poder enviar las modificaciones la segunda vez?*

Finalmente vamos a hablar un poco de las ramas. Una de las grandes virtudes de `git` es que permite muy fácilmente abrir ramas del proyecto y volver a incorporarlas al proyecto principal. Fijaros que en la última orden que hemos visto (`git push origin master`) incorporamos un nombre: `master` que ya solía aparecer en `git status` (si no fijaros ahora). Este nombre es el nombre de la rama de trabajo por defecto. El tener varias ramas sirve para tener varias versiones diferentes en las que trabajar en paralelo de forma que las más exitosas puedan incorporarse a la rama principal y el resto descartarse de forma fácil. A continuación, uno de los grupos, probad a crear una nueva rama:

```
git branch nombrerama
```

Y cambiar a ella: `git checkout nombrerama`. A continuación cambiad cosas en esa rama y que el otro grupo cambie los mismos ficheros pero en la rama principal y haced ambos el `push`.

Pregunta 8.9: *¿Con qué comando se debe hacer el `push` de la nueva rama?*

Finalmente, pasado un cierto tiempo suele haber que volver a mezclar la rama nueva (`git merge nombrerama`, la mezcla con la actual) y, seguramente eliminarla (`git branch -d nombrerama`).

Pregunta 8.10: *¿Qué comandos tiene que ejecutar el que ha creado la nueva rama para volver a mezclarla y eliminarla? ¿Hay algún problema si la nueva rama solo tiene modificaciones propias (que no comparte con la principal)? ¿Y si hay modificaciones en la misma porción del mismo archivo de las dos ramas que se quieren fusionar?*

Fijaros que siempre podéis comprobar las diferencias con la rama principal mediante el comando `git diff HEAD` (`HEAD` es un apuntador a la última versión de la rama principal).

Otra característica importante de `git` es que permite muy fácilmente incorporar acciones automáticas cuando se realizan ciertas acciones. Esto se hace mediante los ganchos (`hooks`). Para ver como funciona id al repositorio remoto inicial que habéis creado (el repositorio `bare`), entrad dentro y mirad el directorio `hooks`. Está lleno de scripts de ejemplo de como usar los ganchos. Básicamente los ganchos son un script con un determinado nombre que,

si existe en ese directorio, se ejecutarán en un momento predeterminado (el momento es, básicamente, el que define su propio nombre). Hay ganchos que se ejecutan en el cliente (es decir, en local) y ganchos que se ejecutan en el servidor (es decir en el directorio remoto). Vamos a ver uno de estos últimos, echadle un vistazo al fichero `pre-receive.sample`). Fijaros que básicamente comprueba los valores de las opciones de `git push` y realiza alguna acción si coinciden con dos valores. Cambiad la opción `reject` por `prueba`.

Pregunta 8.11: *¿Como se pasan opciones a `git push`?*

Ahora vamos a probarlo. Para eso en primer lugar hay que activar el gancho en el servidor. Es tan simple como mover o copiar el fichero `pre-receive.sample` a `pre-receive`. También hay que permitir que el `push` tenga opciones:

```
git config receive.advertisePushOptions true
```

Una vez hecho volved al directorio de trabajo, cread un fichero nuevo, haced el `commit` del fichero y realizad el `push` usando la opción `-o prueba`.

Pregunta 8.12: *¿Que mensaje devuelve el `push`? ¿Se ha actualizado el fichero en el repositorio remoto? ¿Y si volvéis a hacer el `push` sin la opción `reject`? ¿Cómo se activa la otra opción preprogramada en el script `pre-receive`?*

Fijaros que es muy fácil a partir de esto hacer pruebas que, por ejemplo, compilen un documento `LATEX`o `.c` en el repositorio remoto o en local y solo hagan `commit` o `push` si no hay errores. Esto abre la puerta a los sistemas de CI como Jenkins.

Pregunta 8.13: *¿Qué es un sistema de CI? ¿Para que sirve Jenkins?*

Antes de acabar con esta parte, comentaros que hay un montón de información sobre Git en la propia página del proyecto (<http://git-scm.com/>) incluido algún libro muy bueno y algún tutorial. Es imposible ver en una clase como va todo lo que hay sobre Git pero si tenéis curiosidad os animo a echar un vistazo a las herramientas GitLab y SparkleShare.

Pregunta 8.14: *¿Para qué sirve GitLab? ¿Y SparkleShare?*

8.2. Nextcloud

En este apartado vamos a ver un poco como funciona Nextcloud. Nextcloud es una de las varias alternativas libres a Google Drive (como `ownCloud`, proyecto del que deriva o el anteriormente mencionado SparkleShare). Toda la configuración y posibilidades de un sistema así está fuera del alcance de este laboratorio, así que vamos a limitarnos a ver como instalar un servidor para uso personal.

Lo primero que cabe destacar es que la ventaja de este tipo de servidores no es su funcionalidad (ya que Drive, por ejemplo, puede funcionar bastante bien) sino el hecho de que se tiene un control bastante completo sobre los datos propios, y eso, dependiendo del caso, puede ser más o menos importante. Además, tanto Nextcloud como, por ejemplo `ownCloud`, tienen un modelo de negocio que incluye ingresos.

Pregunta 8.15: *¿Cual es el modelo de negocio de Nextcloud? ¿y el de ownCloud?*

A continuación vamos a instalar Nextcloud. Para ello, lo primero que tenemos que hacer es hablar de `snap`. `snap` es un sistema de paquetes que trata de remediar los problemas de compatibilidad comunes al sistema de paquetes estándar. Fijaros que una de las mayores ventajas de los sistemas de paquetes de Linux, el hecho de poder usar unas librerías comunes, es también una debilidad. ¿Qué sucede si una librería se actualiza pero una aplicación que la utiliza no lo hace? ¿Y si dos aplicaciones usan dos versiones distintas (e incompatibles) de la misma librería? Para solucionar esto, `snap` (y otras alternativas competidoras como Flatpack o AppImage) empaquetan las aplicaciones junto con sus dependencias en un solo paquete que puede funcionar de forma bastante independiente a la distribución instalada.

Es casi un contenedor tipo Docker pero sin abstraer el sistema operativo. Evidentemente las diferentes alternativas están soportadas por diferentes distribuciones y hay una lucha de formatos que todavía está por ver quien ganará (similar a lo que sucede con los paquetes .deb y .rpm) si es que alguien gana.

En cualquier caso, Ubuntu incorpora snap por defecto desde hace varios años y cada vez lo integra más con la tienda de aplicaciones. De hecho, Firefox en Ubuntu no corre nativo sino dentro de snap lo que permite que sus actualizaciones se desplieguen más rápido (ya que no hace falta adaptarlas al SO destino). Podéis ver que snaps tiene vuestro sistema instalado tecleando `snap list`. Suponiendo que alguno no esté actualizado (otra de las ventajas de los snaps es que se actualizan automáticamente de forma independiente al sistema apt) podéis actualizarlos con `snap refresh`.

Por citar algún problema de los snaps es que ahora, desde línea de comandos (el gestor gráfico integra apt y snap) hay dos repositorios diferentes de instalación y aunque algunos paquetes están en ambos, lo habitual es que solo estén en uno y tengamos que saber donde buscar. Podéis usar `snap find` para buscar.

Pregunta 8.16: *¿Está el paquete nextcloud (a secas) disponible en apt? ¿Y en snap? ¿Cómo se instala un paquete en snap?*

Ahora y podéis instalar nextcloud en vuestro sistema. Para empezar a usarlo abrid el navegador y conectaros a localhost. Cread una cuenta de administrador e instalad las aplicaciones sugeridas. A continuación cread un documento desde el propio navegador. Fijaros que los ficheros aquí creados aparecen en un fichero dentro del directorio `/var/snap/nextcloud/` y no dentro de `/home` así que si pretendéis poner un servidor con nextcloud es bueno tener el `/var/` montado en una partición separada del disco.

Pregunta 8.17: *¿En que directorio se guardan los ficheros creados con el editor de nextcloud? ¿Usa por defecto un formato libre?*